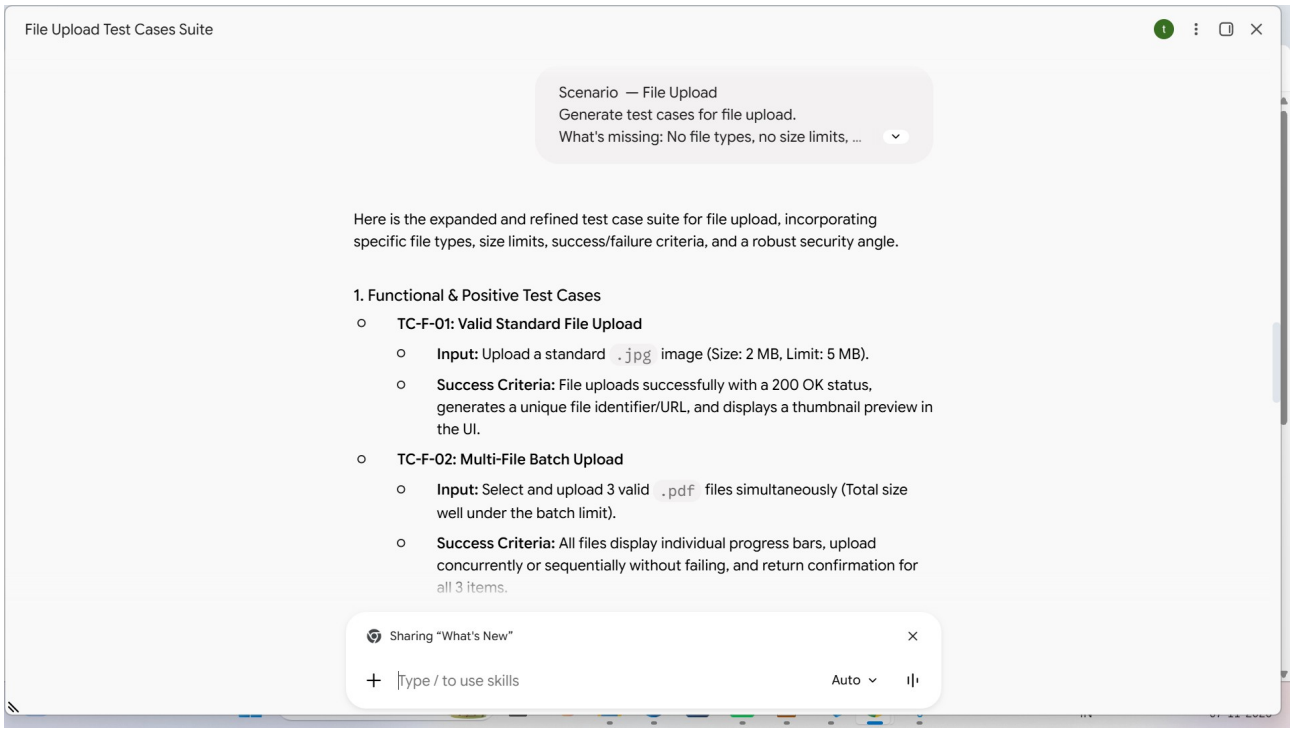
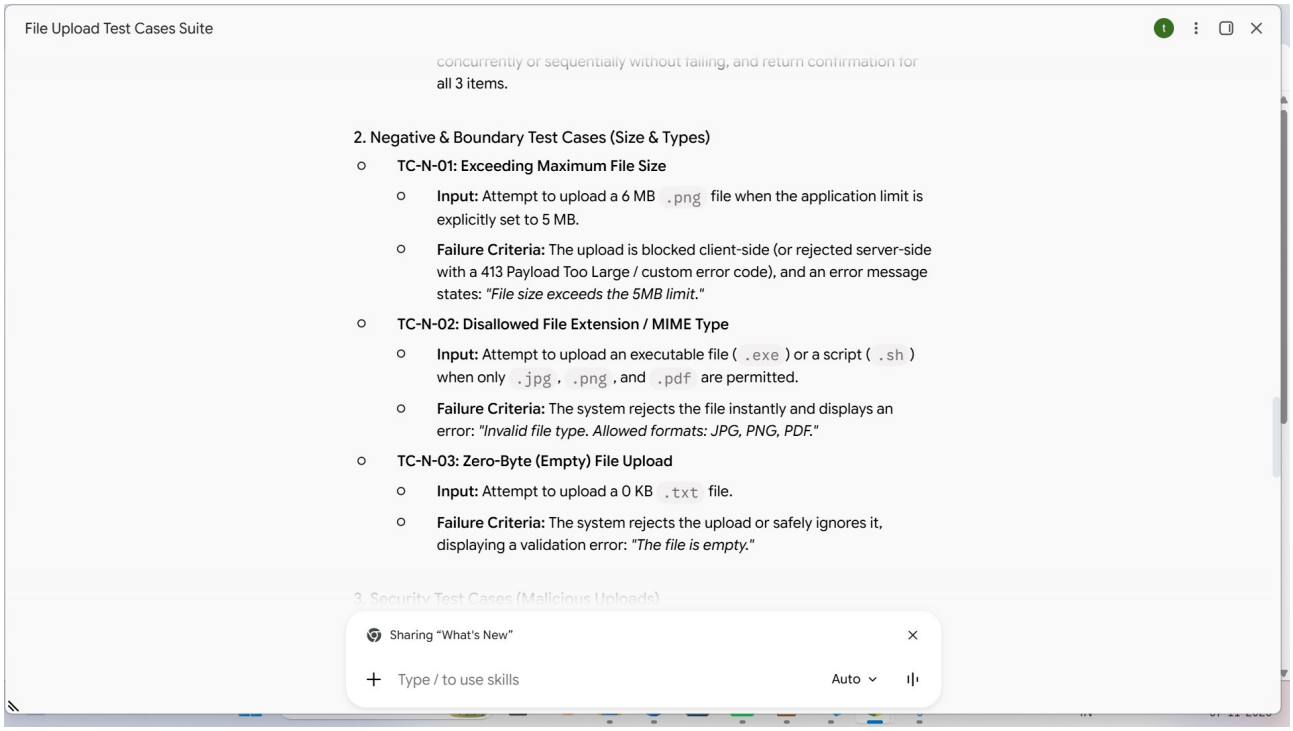
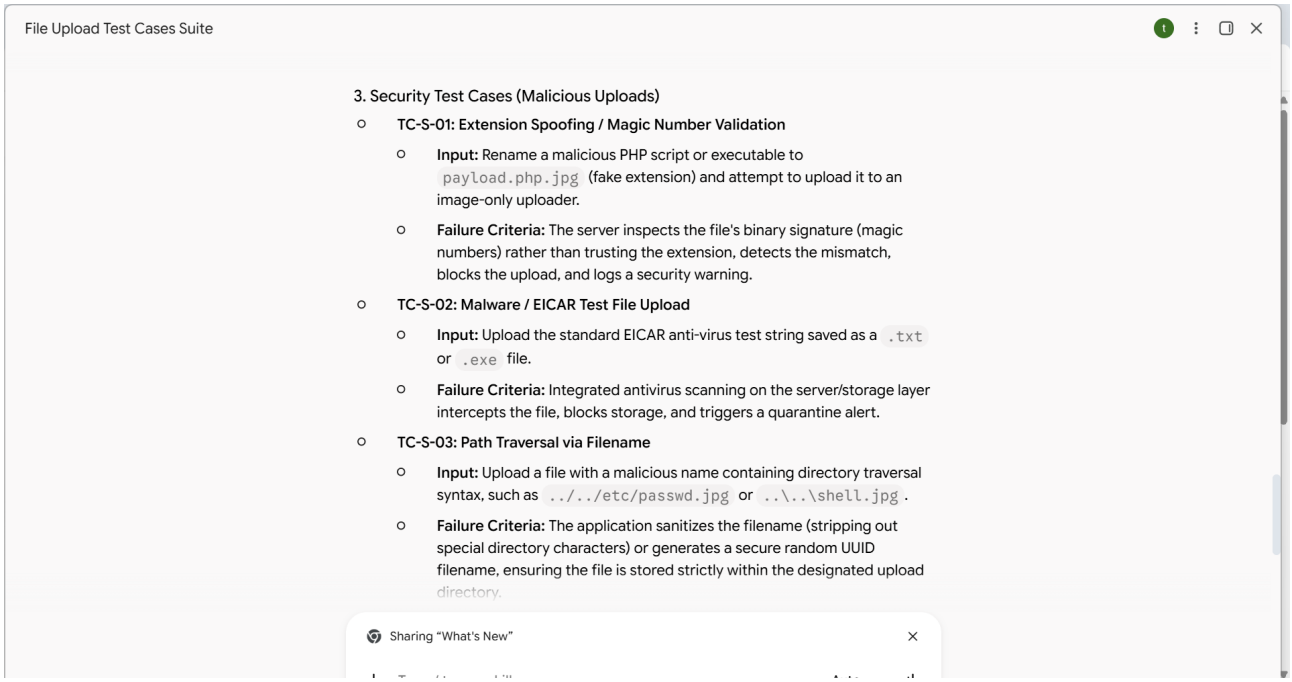
Comparison between Weak prompt and ICEPOT prompt:

1. Generated test case was not clear and structured.
2. Pre-condition for test cases is missing.
3. Test steps for test cases were not generated from weak prompt.

Scenario 2 - File Upload Test cases for Weak Prompt:







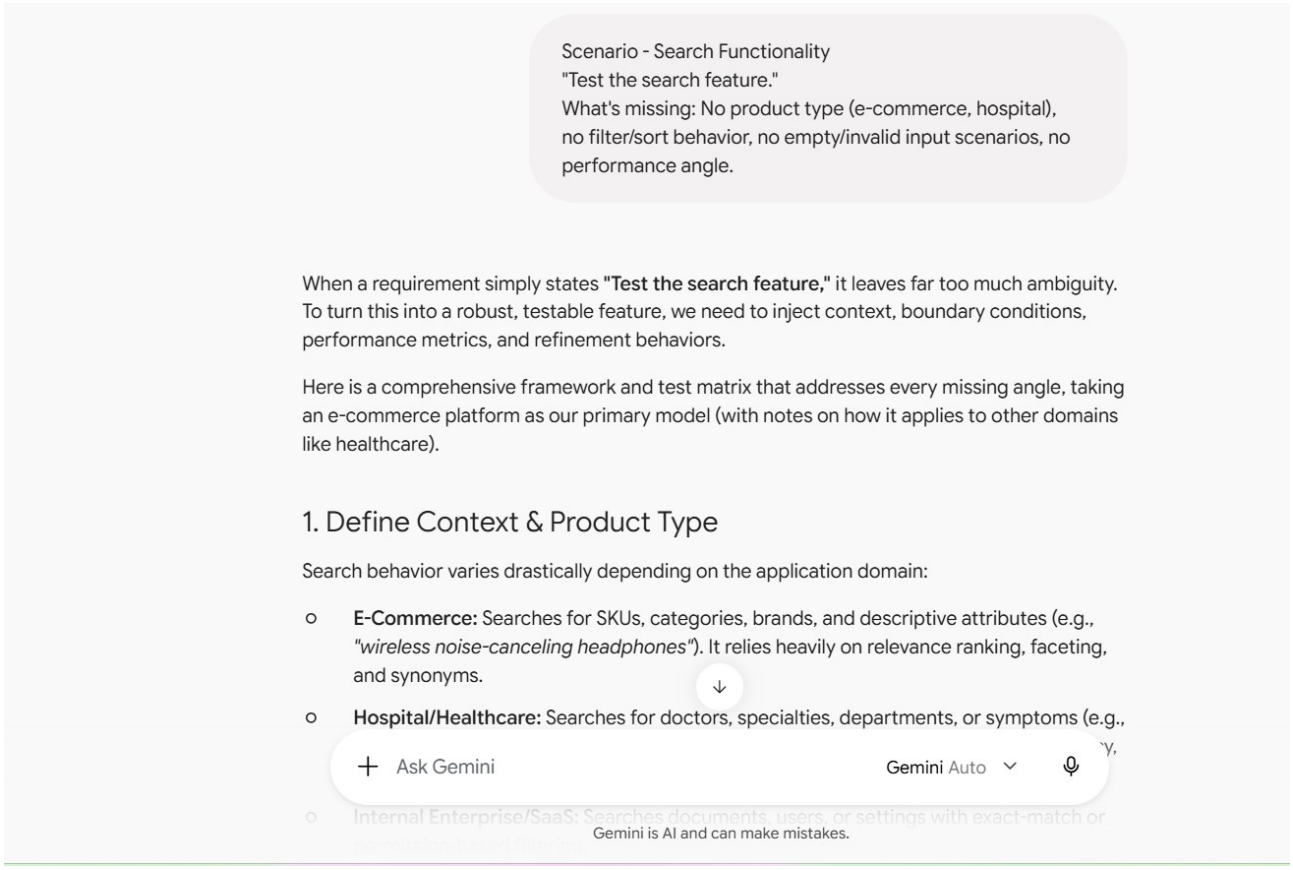
Scenario 2 - File Upload Test cases for ICEPOT Prompt:

Test case ID	Description	Test Data	Pre Condition	Test Steps	Expected Result
TC_001	Verify successful file upload with valid format and size (Positive)	File: profile.jpg (Format: JPG, Size: 2 MB, Limit: 5 MB)	User is logged in and navigates to the file upload page.	"1. Click on the Browse/Upload button 2. Select a valid JPG file 3. Click the Submit/Upload button"	File uploads successfully, displays a success confirmation message, and shows a thumbnail preview.
TC_002	Verify file upload	File: large_doc	User is logged in	"1. Click on the	Upload is blocked/rejecte

Test case ID	Description	Test Data	Pre Condition	Test Steps	Expected Result
	rejection when exceeding maximum size limit (Negative)	ument.pdf (Size: 8 MB, Limit: 5 MB)	and navigates to the file upload page.	Browse/Upload button 2. Select a file exceeding the size limit 3. Click the Submit/Upload button"	d, and an error message displays: *"File size exceeds the 5MB limit."*
TC_003	Verify file upload rejection for an unsupported file format (Negative)	File: script.exe (Format: EXE, Limit: JPG, PNG, PDF)	User is logged in and navigates to the file upload page.	"1. Click on the Browse/Upload button 2. Select an unsupported file extension 3. Click the Submit/Upload button"	Upload is rejected, and an error message displays: *"Invalid file type. Allowed formats: JPG, PNG, PDF."*
TC_004	Verify system behavior when attempting to upload an empty file (Edge Case)	File: empty.txt (Size: 0 KB)	User is logged in and navigates to the file upload page.	"1. Click on the Browse/Upload button 2. Select a 0 KB empty file	System catches the empty file validation and displays an error message: *"The file is empty and cannot be uploaded."*

Test case ID	Description	Test Data	Pre Condition	Test Steps	Expected Result
				3. Click the Submit/Upload button"	
TC_005	Verify file upload behavior with special characters in the filename (Edge Case)	File: my resume (final)_#2.pdf (Valid format and size)	User is logged in and navigates to the file upload page.	"1. Click on the Browse/Upload button 2. Select a file containing spaces and special characters 3. Click the Submit/Upload button"	File uploads successfully, and the system safely sanitizes or encodes the filename in storage and UI rendering.

Scenario 1 – Search Functionality Test cases for Weak Prompt:



2. Input Validation & Edge Cases

You must test how the search engine handles normal, malformed, malicious, and empty inputs:

Category	Test Scenario	Expected Behavior
Empty Input	Clicking search with an empty input field or whitespace.	Prevent submission, display a helpful prompt ("Please enter a search term"), or show recent/trending searches.
Special Characters & SQL/XSS	Entering symbols like `!@#$%^&*()` or injection strings like `<script>alert(1)</script>`.	System must sanitize input, prevent code execution, and safely return "No results found" or error gracefully without crashing.
Long Strings	Entering a 500-character string or paragraph.	Input field should truncate or wrap; the backend should handle long payloads without timing out (e.g., 504 Gateway Timeout).
Typ		

Ask Gemini Gemini Auto
Gemini is AI and can make mistakes.

3. Filtering, Sorting, and Faceted Search

A search feature is rarely just a text box; users expect to refine their results dynamically:

- **Sorting Logic:** Test sorting by **Relevance**, **Price (Low to High / High to Low)**, **Rating**, and **Newest Arrivals**. Verify that pagination or infinite scroll resets or maintains state correctly when sorting changes.
- **Multi-Select Filters:** Apply multiple facets simultaneously (e.g., Brand = *Sony* AND Price = *$100-$200* AND In-Stock = *True*). Verify that filter counts update dynamically and zero-result combinations disable or display clearly.
- **Clear Filters:** Test the "Clear All" or individual filter removal buttons to ensure results revert instantly to the unfiltered state.

4. Performance, Latency, and Load

Search is often the most resource-intensive query on a database. Performance criteria must be explicitly verified:

- **Debouncing & Throttling:** As the user types in the autocomplete dropdown, verify that requests are debounced (e.g., firing only after a 300ms pause) rather than firing a request on every single keystroke.
- **Latency (Response Time):** Autocomplete suggestions should load in under **100–200ms**, while full search result pages should ren within **1–2 seconds** under normal load.
- **High Concurrency / Load Testing:** Simulate hundreds of simultaneous users performing

Ask Gemini Gemini Auto
Gemini is AI and can make mistakes.

explicitly verified:

- **Debouncing & Throttling:** As the user types in the autocomplete dropdown, verify that requests are debounced (e.g., firing only after a 300ms pause) rather than firing a request on every single keystroke.
- **Latency (Response Time):** Autocomplete suggestions should load in under **100–200ms**, while full search result pages should render within **1–2 seconds** under normal load.
- **High Concurrency / Load Testing:** Simulate hundreds of simultaneous users performing complex searches to check for database connection pool exhaustion or search index lag (using tools like JMeter or k6).

5. UI/UX and Accessibility

- **State Indicators:** Verify that a loading spinner or skeleton loader appears immediately while fetching results.
- **Empty State Design:** Ensure a clean "No results found for [query]" screen is displayed, ideally accompanied by alternative suggestions, trending categories, or a prompt to clear filters.
- **Keyboard Navigation:** Users should be able to type a query, use the `Arrow Down` / `Arrow Up` keys to navigate autocomplete dropdown suggestions, and press `Enter` to select.

Want me to convert this into a Gherkin/BDD test script suite (Given/When/Then) for a

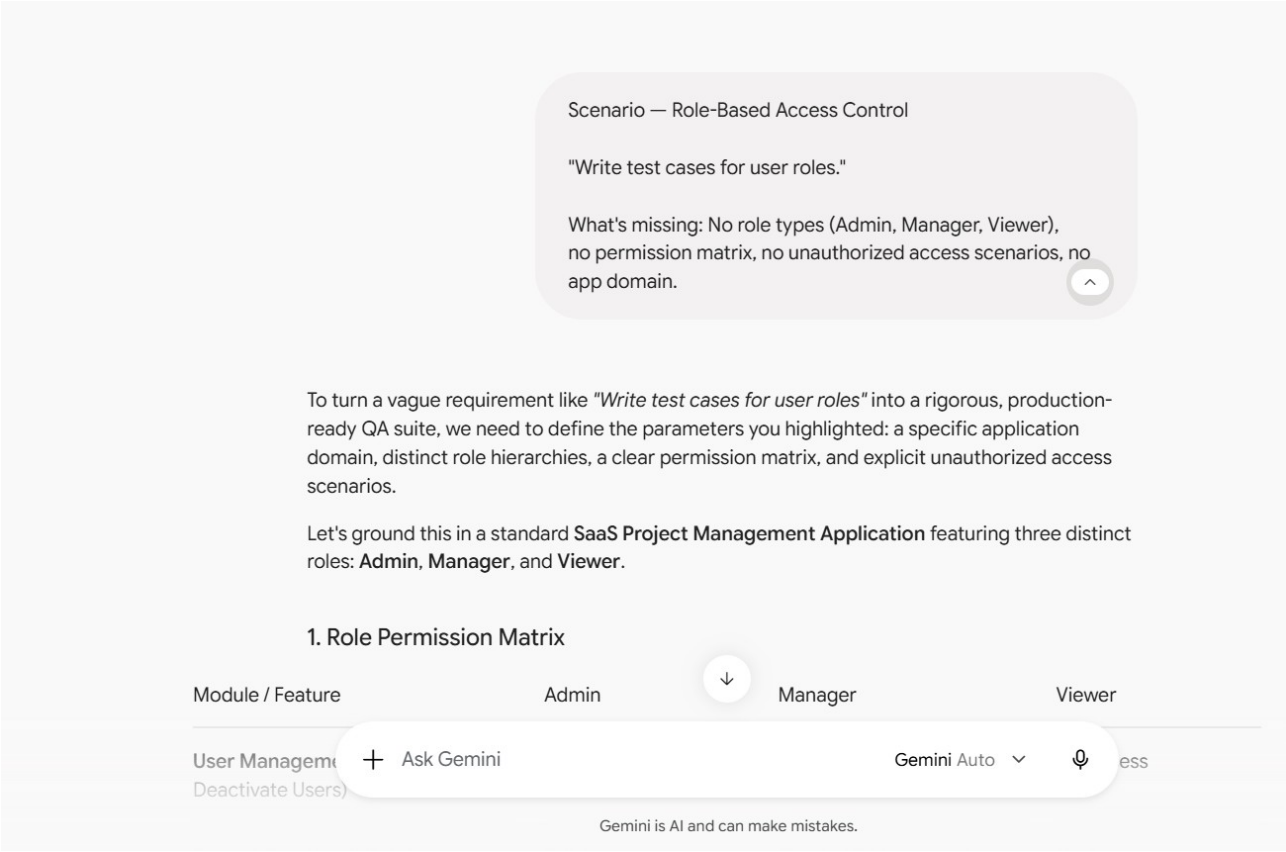
Ask Gemini Gemini Auto

Gemini is AI and can make mistakes.

Scenario 1 – Search Functionality Test cases for ICEPOT Prompt:

Test Case ID	Description	Category	Test Data	Pre-Condition	Test Steps	Expected Result
TC_SEARCH_001	Verify search with valid alphanumeric keywords (Positive Scenario)	Positive	Search Keyword: "Laptop 15"	Application homepage is loaded and search bar is visible and enabled.	1. Navigate to the main page. 2. Enter "Laptop 15" into the search bar. 3. Click the Search button or press Enter.	System displays matching search results with relevant headers and correct count; search input term is retained in the search box.
TC_SEARCH_002	Verify search behavior with non-matching/invalid keyword (Negative Scenario)	Negative	Search Keyword: "xyz999888nonexistent"	Application homepage is loaded and search index is accessible.	1. Enter "xyz999888nonexistent" in the search field. 2. Click the Search button.	System displays a clear visual message: "No results found for 'xyz999888nonexistent'" along with recommended next steps or category links without throwing application errors.
TC_SEARCH_003	Verify empty input submission handling (Edge Scenario)	Edge	Search Keyword: " " (Spaces only / Blank)	Application search field is active.	1. Click into the search bar. 2. Enter multiple spaces (or leave completely blank). 3. Click the Search icon/button.	Search request is prevented from executing; system displays an inline validation message "Please enter a search term" or keeps the user on the current page.
TC_SEARCH_004	Verify search input with special characters & potential script tags (Edge Scenario)	Edge / Security	Search Keyword: "<script>alert('test')</script> &@#$%"	Search input field is loaded.	1. Paste script tag and special characters into the search field. 2. Execute search.	Input is sanitized properly; script does not execute, no XSS vulnerability is triggered, and a safe "No results found" or sanitized string result page is displayed.
TC_SEARCH_005	Verify search input boundary length limit (Edge Scenario)	Edge	Search Keyword: 256+ character string ("A" * 300)	Application homepage is loaded.	1. Enter a string exceeding max character limit (e.g., 300 characters). 2. Click Search.	Input field either truncates input at maximum allowed limit (e.g., 255 characters) or handles request gracefully without application crash, 500 error, or UI distortion.

Scenario 4 – Role-Based Access Control test cases for weak prompt:



2. Positive Test Cases (Authorized Access)

These test cases verify that users can successfully perform actions permitted by their designated role.

- **TC_RBAC_POS_001: Admin Global Access**
  - **Pre-condition:** User is logged in with an *Admin* role.
  - **Test Steps:** Navigate to System Settings, Billing, User Management, and Project Dashboards.
  - **Expected Result:** Admin can access, modify, and save configurations across all modules without restriction.
- **TC_RBAC_POS_002: Manager Project Control**
  - **Pre-condition:** User is logged in with a *Manager* role assigned to Project Alpha.
  - **Test Steps:** Create a new task within Project Alpha, assign it to a team member, and update task status.
  - **Expected Result:** Actions are successfully saved and reflected in the project log. Manager cannot access Billing or global system settings.
- **TC_RBAC_POS_003: Viewer Read-Only Verification**
  - **Pre-condition:** User is logged in w *Viewer* role.
  - **Test Steps:** Open Project Alpha and attempt to locate "Create Task", "Edit", or

Ask Gemini Gemini Auto

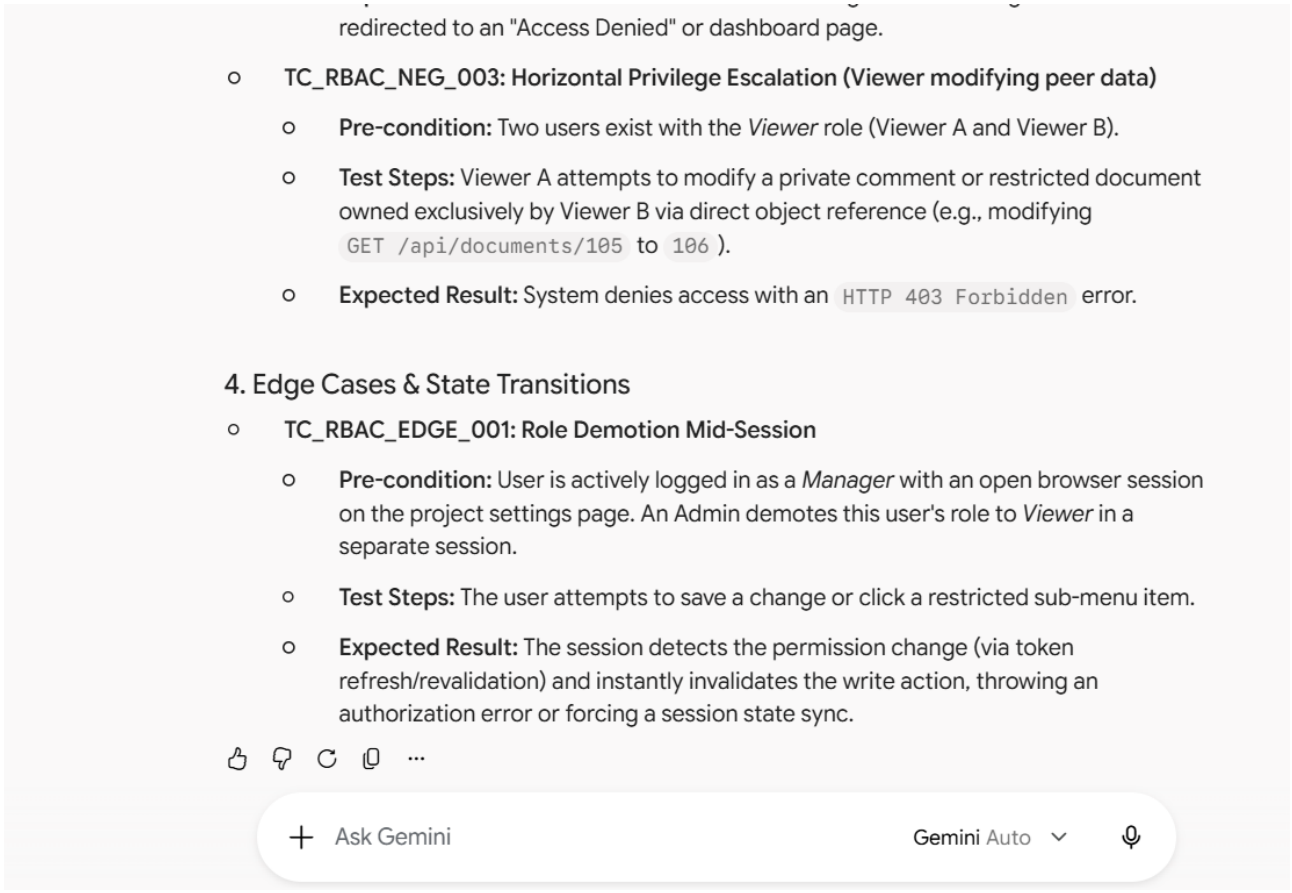
controls (Create, Edit, Delete, Settings) are either hidden or disabled.

3. Negative Test Cases (Unauthorized Access & Privilege Escalation)

These test cases ensure the system properly blocks unauthorized attempts and prevents privilege escalation.

- **TC_RBAC_NEG_001: Viewer Attempting Task Creation**
  - **Pre-condition:** User is logged in as a *Viewer*.
  - **Test Steps:** Attempt to bypass the UI restriction by sending a direct `POST /api/v1/projects/{id}/tasks` API request using a tool like Postman or browser developer tools.
  - **Expected Result:** The server returns an `HTTP 403 Forbidden` (or `401 Unauthorized`) response, and the task is not created in the database.
- **TC_RBAC_NEG_002: Manager Accessing Billing Module**
  - **Pre-condition:** User is logged in as a *Manager*.
  - **Test Steps:** Directly enter the URL route `/settings/billing` in the browser address bar.
  - **Expected Result:** The user is block om viewing sensitive billing data and redirected to an "Access Denied" or dashboard page.
-

Ask Gemini Gemini Auto



Scenario 4 – Role-Based Access Control Test cases for ICEPOT Prompt:

Test Case ID	Description	Test Data	Pre Condition	Test Steps	Expected Result
TC_RBAC_001	Verify Admin user has full access to create, edit, and delete user roles (Positive)	Role: Admin Username: admin_user@app.com Password: Password@123 Target Role: Manager	1. Application URL is loaded. 2. User with 'Admin' role is logged in and on User Management module.	1. Click on 'Create New User' button. 2. Enter valid user details and select Role as 'Manager'. 3. Click 'Save'. 4. Select newly created user and click 'Edit Role'. 5. Change role to 'Viewer' and click 'Update'. 6. Click 'Delete User'.	1. User with assigned role is created successfully. 2. Role is updated to 'Viewer' without errors. 3. User record is deleted successfully and appropriate success messages are displayed.
TC_RBAC_002	Verify Manager user can access assigned module features but cannot access Admin settings (Positive)	Role: Manager Username: manager_user@app.com Password: Password@123 Assigned Project: Project Alpha	1. Application URL is loaded. 2. User with 'Manager' role is successfully authenticated.	1. Navigate to Project Management dashboard. 2. Create a new task within assigned 'Project Alpha'. 3. Attempt to locate 'User Roles Management' or 'System Settings' in navigation menu.	1. Task is created and saved successfully within Project Alpha. 2. 'User Roles Management' and 'System Settings' menus/buttons are hidden or absent for Manager role.
TC_RBAC_003	Verify Viewer role user cannot perform write operations or see action controls (Positive/Negative)	Role: Viewer Username: viewer_user@app.com Password: Password@123 Target Page: Project Dashboard	1. Application URL is loaded. 2. User with 'Viewer' role is logged into the application.	1. Navigate to Project Dashboard. 2. Observe available UI controls on the page. 3. Attempt to view project details and task lists.	1. All project data is displayed in Read-Only mode. 2. Action controls such as 'Create Task', 'Edit', 'Delete', and 'Save' buttons are hidden/disabled.
TC_RBAC_004	Verify unauthorized access prevention via direct URL navigation for restricted roles (Negative)	Role: Viewer Username: viewer_user@app.com Password: Password@123 Restricted Route: /admin/user-management	1. Application URL is loaded. 2. User is logged in as 'Viewer' role.	1. Open a new tab or use current address bar. 2. Directly enter restricted admin URL: '/admin/user-management'. 3. Press Enter.	1. Access is denied. 2. System redirects user to 'Access Denied / 403 Forbidden' page or main dashboard with proper error message.
TC_RBAC_005	Verify session privilege updates when user role is updated dynamically mid-session (Edge Case)	Roles: Manager to Viewer transition Username: user_temp@app.com Password: Password@123	1. User 'user_temp' is logged in as 'Manager' on Browser 1. 2. Admin is logged in on Browser 2.	1. On Browser 1, User opens project edit form. 2. On Browser 2, Admin updates 'user_temp' role from 'Manager' to 'Viewer'. 3. On Browser 1, User clicks 'Save' on project edit form.	1. System validates session rights upon API submission. 2. Request is rejected with HTTP 403 / Access Revoked response. 3. Session token/state updates UI immediately to Read-Only mode.